UNIVERSIDAD DE LOS ANDES
ISIS-1226 · DISEÑO Y PROGRAMACIÓN ORIENTADA A OBJETOS

ANEXO DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA

Sistema de analítica deportiva del Club Deportivo Séneca
Proyecto 1 — Entrega 2

1. ALCANCE DEL USO

El enunciado exige documentar el uso de herramientas de inteligencia artificial generativa, incluyendo la colección de prompts empleados. Esta sección recoge el uso efectivo que hizo el grupo durante la Entrega 2, antes de subir la entrega al repositorio.

2. HERRAMIENTAS UTILIZADAS

Herramienta	**Uso dado**	**Entregable sobre el que se usó**
Claude (Anthropic)	Sugerir y justificar el uso de Map<String, X> (LinkedHashMap) para buscar personas por documento y equipos por código sin recorrer listas ni admitir duplicados.	Club.java, CatalogoReglas.java
Claude (Anthropic)	Explicar el uso de enum frente a una clase con constructor privado y constantes estáticas, como alternativa para representar Categoria y Nivel.	Categoria.java, Nivel.java
Claude (Anthropic)	Justificar el uso de java.time (LocalDate, LocalTime) como biblioteca estándar y delimitar los métodos usados (isBefore, isAfter, parse, now, Period.between).	Clases del modelo que manejan fechas y horas (persistencia, modelo.personas, entre otras)
Claude (Anthropic)	Aplicar correcciones puntuales a partir del informe de revisión de código: documentar enum y java.time, corregir el comando de compilación del README para PowerShell y precisar que el repositorio recibe la ruta de datos por constructor.	README.txt, documento de trazabilidad de requerimientos
Claude (Anthropic)	Contrastar los diagramas de secuencia contra el código método por método y revisar el diagrama de clases para detectar asociaciones de Club no representadas.	Diagrama de clases, diagramas de secuencia

3. COLECCIÓN DE PROMPTS

Transcribir aquí los prompts efectivamente utilizados, agrupados por entregable.

3.1 Diseño e implementación

• El Club tiene que poder buscar una persona por su documento y un equipo por su código sin recorrer una lista entera cada vez. ¿Qué me recomiendas usar y por qué?

• ¿Qué es exactamente un Map en Java y en qué se diferencia de una lista? ¿Y qué hace distinto un LinkedHashMap? Explícamelo sencillo porque no lo hemos visto en clase.

• La categoría de un equipo y el nivel de un afiliado son listas cerradas de valores de las que dependen validaciones. ¿Me conviene guardarlas como texto o como enum? Explícame qué es un enum y qué pasaría si lo hiciera con texto.

• ¿Puedo usar LocalDate y LocalTime en un proyecto donde no se permiten librerías externas? Dime qué hace cada método que necesite y por qué no me sirve guardar las fechas como texto.

• El comando javac -d bin $(find src -name "*.java") me falla en PowerShell. Dame la versión que funcione en Windows y explícame por qué cambia.

3.2 Documentación

• Revisa la ortografía y la redacción de este documento y corrígela donde haga falta.

4. REVISIÓN DEL GRUPO

Describir aquí qué se revisó, qué se corrigió y qué se reescribió del material generado antes de incorporarlo

a la entrega. Durante la Entrega 2 usamos Claude como apoyo y estos son los puntos donde su aporte

cambió el resultado.

Elección del Map. El Club tenía que poder buscar una persona por su documento y un equipo por su código sin recorrer una lista entera cada vez, y sin que se repitiera el identificador. Estructuras de diccionario las habíamos visto en un semestre anterior pero en Python, no en Java, entonces le preguntamos a Claude qué nos recomendaba usar y por qué. Nos explicó que un Map en Java es justamente eso, una estructura que guarda pares de clave y valor, que la clave no se puede repetir porque así funciona por construcción, y que si además usábamos LinkedHashMap el orden en el que íbamos agregando los datos se mantenía, cosa que nos servía para que los reportes salieran siempre igual. Con esa explicación decidimos usar Map<String, X>, en concreto LinkedHashMap, y dejamos esa misma justificación como comentario dentro de Club y de CatalogoReglas.

Uso de enum. La categoría de un equipo y el nivel de un afiliado son listas cerradas de valores de las que dependen validaciones, entonces la duda era si guardarlas como texto o como enum, y ahí tampoco teníamos nada visto en clase. Claude nos explicó qué es un enum, que es una clase con un conjunto fijo de instancias que ya vienen creadas de antemano, y que lo mismo se podría hacer con una clase de constructor privado y constantes estáticas pero sin ese mismo control. Nos convenció el enum porque cada valor lleva su propio dato, el rango de edad en Categoria y el orden en Nivel, y porque si alguien escribe mal el valor el compilador no deja compilar, mientras que con texto libre eso se cuela sin que nadie se dé cuenta.

Uso de java.time. También le preguntamos si podíamos usar LocalDate y LocalTime en un proyecto donde no se permiten librerías externas, porque no sabíamos si contaban como tal. Nos explicó que java.time es parte de la biblioteca estándar de Java, entonces no es una librería externa, y nos dijo qué hacía cada método que necesitábamos: isBefore e isAfter para comparar fechas, parse para leerlas desde el archivo, now para la fecha actual y Period.between para sacar una diferencia de tiempo. Eso fue lo que acotamos usar, nada más, porque si guardábamos las fechas como texto no podíamos compararlas ni hacer esas cuentas directamente.

Correcciones a partir del documento de revisión de código. Le entregamos a Claude el informe de hallazgos de la revisión de código para que nos ayudara a aplicar las correcciones, y de los seis hallazgos aplicó cuatro: documentar el enum y java.time, corregir el comando de compilación del README porque tenía sintaxis de Linux y en PowerShell no corría, precisar en la trazabilidad que el repositorio recibe la ruta de datos por constructor y no la tiene fija, y actualizar la división del trabajo del grupo. Los otros dos hallazgos, la falta de clases de consola y el anexo de IA, esos los resolvimos nosotros mismos.

Verificación de los diagramas. Por último, Claude revisó los diagramas de secuencia contra el código método por método y ahí encontró que al diagrama de clases le faltaban cinco asociaciones de Club. Lo que pasaba era que el diagrama se genera leyendo los atributos del código y la lectura de tipos genéricos solo reconocía los de un parámetro, entonces las colecciones que habíamos declarado como Map<String, X> se quedaban sin aparecer. Lo corregimos y el modelo pasó de tener 91 relaciones a tener 96.